

Robot Kinematics Analysis Theory

I. Experiment Objective

Understand the kinematics model of the differential-drive robot chassis, and master the forward/inverse kinematics solving methods from the left and right wheel speeds to the overall linear velocity and angular velocity of the robot. By reading the source code, understand how to implement robot motion control functions on the ESP32-S3, and how to control the car's motion state through interfaces such as `Motion_Ctrl()` and `Motion_Ctrl_State()`.

II. Experiment Principle

1. Keywords

Keyword	Explanation
Differential drive	A chassis structure with two independently driven wheels that steers by the speed difference between the left and right wheels. When the left wheel speed < the right wheel speed, it turns left; when the left wheel speed > the right wheel speed, it turns right; when the speeds are equal in magnitude and opposite in direction, it rotates in place.
Forward kinematics	Given the actual speeds of the left and right wheels, deduce the overall linear velocity V_x and angular velocity W_z of the chassis. Formulas: $V_x = (V_{left} + V_{right}) / 2$, $W_z = -(V_{left} - V_{right}) / \text{wheel track}$.
Inverse kinematics	Given the target linear velocity V_x and angular velocity W_z , calculate the target speeds of the left and right wheels. Formulas: $V_{left} = V_x - W_z * \text{half wheel track}$, $V_{right} = V_x + W_z * \text{half wheel track}$.
Odometry	A basic localization method that accumulates encoder pulses to calculate the robot's traveled distance and heading change. Kinematics solving is the core mathematical basis of odometry.
Turning radius	The radius of the circular arc trajectory when the robot turns. In this experiment, the turning radius is fixed at 0.30m, configured through the <code>ROBOT_TURN_RADIUS</code> parameter, and the turning scale coefficient is $1/0.30 \approx 3.333$.

2. Technical Architecture

Building on PID motor control (Section 16.11), the program adds the `car_motion` kinematics solving layer, which encapsulates the forward/inverse kinematics calculation of the differential chassis and provides high-level motion control interfaces such as `Motion_Ctrl`, `Motion_Get_Speed`, and `Motion_Ctrl_State`.

```

main/main.c
└─ Motion_Ctrl(Vx, Vy, Wz) / Motion_Ctrl_State(state, speed)
    └─ components/car_motion/      Kinematics solving layer (inverse → wheel speeds /
forward → chassis velocity)
        └─ components/motor/      Reuse PID motor control (Section 11)
        └─ components/encoder/    Reuse the encoder driver
        └─ components/pwm_motor/  Reuse the PWM motor driver

```

Main control flow: call `Motion_Ctrl(1.0, 0, 0)` to set the linear velocity → inverse kinematics calculates the target speeds of the left and right wheels → call `Motor_Set_Speed` to drive the motors → PID closed-loop adjustment → encoder feedback → forward kinematics calculates the actual chassis velocity.

III. Experiment Steps

1. Hardware Preparation

Hardware	Requirement
MicroROS-V2 control board	✓ Required
USB data cable (Type-C)	✓ Required
Motor (with encoder)	✓ Required (built into the chassis)
Battery (12V)	✓ Required (chassis power supply)

2. Prerequisites

1. Set up the ESP32-IDF development environment (see Chapter 15).
2. Complete Section 16.11, PID Motor Speed Control, and understand the basic flow of motor closed-loop control
3. Connect the MicroROS-V2 control board to the computer via USB
4. The motor power supply is normal

3. Compile and Flash

```

cd ~/esp/base_samples/car_motion
source ~/esp/esp-idf/export.sh
idf.py build flash monitor

```

To exit the monitor: press `Ctrl + ]`.

4. Stopping Method

In the terminal, press `Ctrl+]` to exit the serial monitor.

IV. Experiment Result

After flashing and running, you can see the following output in the serial monitor terminal:

```
hello yahboom
I (1012) MAIN: Nice to meet you!
I (1022) MOTOR: Start Motor_Task with core:1
I (1022) MAIN: Read Motion:0.28, 0.00, 0.02
I (1122) MAIN: Read Motion:0.52, 0.00, -0.01
I (1222) MAIN: Read Motion:0.73, 0.00, 0.01
I (1322) MAIN: Read Motion:0.89, 0.00, -0.01
I (1422) MAIN: Read Motion:0.96, 0.00, 0.00
I (1522) MAIN: Read Motion:0.99, 0.00, 0.00
```

- V_x gradually accelerates from 0 to near the set 1.0 m/s
- V_y is always 0 (the differential chassis has no Y-direction velocity)
- W_z is near 0 (straight-line motion, theoretically no rotation)
- About 20 seconds later, the motor coasts to a stop

V. Program Analysis

Source code paths:

- Firmware source: `~/esp/base_samples/car_motion/main/main.c`
- Kinematics solving: `~/esp/base_samples/car_motion/components/car_motion/car_motion.h`, `car_motion.c`
- Motor PID: `~/esp/base_samples/car_motion/components/motor/motor.h`, `motor.c`

1. Directory Structure

```
car_motion/
├── CMakeLists.txt
├── main/
│   ├── CMakeLists.txt
│   └── main.c
├── components/
│   ├── car_motion/           # kinematics solving layer
│   │   ├── car_motion.c
│   │   └── car_motion.h
│   ├── motor/               # Reuses the motor control from Section 11
│   ├── encoder/             # Reuses the encoder from Section 11
│   └── pwm_motor/           # Reuses the PWM motor driver from Section 11
```

The current main program only demonstrates motion control.

2. Main Program Logic

`main/main.c` implements a straight-line motion test:

1. Delay 1 second to wait for the system to stabilize
2. Call `Motor_Init()` to initialize the motor system
3. Call `Motion_Ctrl(1.0, 0, 0.0)` to set the linear velocity to 1.0 m/s and the angular velocity to 0 rad/s, i.e., moving straight forward
4. Loop to read and print the car's current linear velocity and angular velocity
5. After running 200 cycles (about 20 seconds), call `Motion_Stop(STOP_COAST)` to stop

```
Motor_Init();
Motion_Ctrl(1.0, 0, 0.0);    // Linear velocity 1.0 m/s, angular velocity 0 rad/s

while (1) {
    Motion_Get_Speed(&micro_car);
    ESP_LOGI(TAG, "Read Motion: %.2f, %.2f, %.2f",
               micro_car.Vx, micro_car.Vy, micro_car.Wz);
    vTaskDelay(pdMS_TO_TICKS(100));
    count++;
    if(count >= 200) {
        Motion_Stop(STOP_COAST);
    }
}
```

3. Key Components

car_motion layer (kinematics model)

Files: `components/car_motion/car_motion.h`, `components/car_motion/car_motion.c`

Physical parameters:

Parameter definition file: `~/esp/base_samples/car_motion/components/car_motion/car_motion.h`

Macro	Value	Description
<code>ROBOT_WIDTH</code>	0.1714f	Distance between the centers of the left and right wheels (171.40mm)
<code>ROBOT_HALF_WIDTH</code>	0.0857f	Half wheel track (85.70mm)
<code>ROBOT_MAX_LINEAR_SPEED</code>	1.0f	Maximum linear velocity (m/s)
<code>ROBOT_MAX_ANGULAR_SPEED</code>	6.0f	Maximum angular velocity (rad/s)
<code>ROBOT_TURN_RADIUS</code>	0.30f	Turning radius (m)
<code>ROBOT_TURN_SCALE</code>	3.333f	Turning scale coefficient (= 1/0.30)
<code>ROBOT_SPIN_MAX_WZ</code>	3.0f	Maximum in-place spin angular velocity (rad/s)

Motion state enum:

```
typedef enum {  
    MOTION_STOP,          // Stop (coast)  
    MOTION_RUN,           // Straight forward  
    MOTION_BACK,         // Straight backward  
    MOTION_LEFT,         // Turn left while moving forward  
    MOTION_RIGHT,        // Turn right while moving forward  
    MOTION_SPIN_LEFT,    // Spin left in place  
    MOTION_SPIN_RIGHT,   // Spin right in place  
    MOTION_BRAKE,        // Brake  
} motion_state_t;
```

Data structure:

```
typedef struct {  
    float vx; // Linear velocity in the X direction (m/s), positive means forward  
    float vy; // Linear velocity in the Y direction (m/s), always 0 for a differential  
    chassis  
    float wz; // Angular velocity around the Z axis (rad/s), positive means  
    counterclockwise  
} car_motion_t;
```

Detailed Kinematics Model

The car is a two-wheel differential-drive chassis. The left and right wheels are driven independently, and steering is achieved by the speed difference between the left and right wheels.

Inverse kinematics (target velocity -> wheel speeds):

Given the target linear velocity `vx` and angular velocity `wz`, calculate the target speeds of the left and right wheels:

```
v_left  = vx - wz * (ROBOT_WIDTH / 2)  
v_right = vx + wz * (ROBOT_WIDTH / 2)
```

where `ROBOT_WIDTH / 2 = 0.0857m` is the half wheel track.

Code implementation (the `Motion_Ctrl` function):

```
speed_L_setup = line_v - angular_v * ROBOT_HALF_WIDTH;  
speed_R_setup = line_v + angular_v * ROBOT_HALF_WIDTH;  
Motor_Set_Speed(speed_L_setup, speed_R_setup);
```

- When $V_z = 0$, the left and right wheel speeds are equal, and the car moves straight forward/backward
- When $V_z > 0$, the left wheel speed $<$ the right wheel speed, and the car turns left
- When $V_x = 0$ and $V_z \neq 0$, the left and right wheel speeds are equal in magnitude and opposite in direction, and the car rotates in place

The speed values are automatically limited to ± 1.0 m/s.

Forward kinematics (wheel speeds -> actual velocity):

Given the actual linear velocities `speed_m1` (left wheel) and `speed_m2` (right wheel), calculate the actual linear and angular velocities of the chassis:

```
vx = (v_left + v_right) / 2
wz = -(v_left - v_right) / ROBOT_WIDTH
```

Note the negative sign in the angular velocity formula. This is because the positive direction of the angular velocity is defined as counterclockwise, while the encoder direction sign configuration makes an increase in the right wheel speed correspond to clockwise rotation.

Code implementation (the `Motion_Get_Speed` function):

```
car->vx = (speed_m1 + speed_m2) / 2.0f;
car->vy = 0;
car->wz = -(speed_m1 - speed_m2) / (ROBOT_WIDTH);
```

Motion state control:

The `Motion_Ctrl_State` function encapsulates common motion control modes. You only need to pass the state enum and the speed magnitude:

Mode	Actual Call	Effect
<code>MOTION_RUN</code>	<code>Motion_Ctrl(speed, 0, 0)</code>	Straight forward
<code>MOTION_BACK</code>	<code>Motion_Ctrl(-speed, 0, 0)</code>	Straight backward
<code>MOTION_LEFT</code>	<code>Motion_Ctrl(speed, 0, speed * 3.333)</code>	Turn left while moving forward with a turning radius of 0.3m
<code>MOTION_RIGHT</code>	<code>Motion_Ctrl(speed, 0, -speed * 3.333)</code>	Turn right while moving forward with a turning radius of 0.3m
<code>MOTION_SPIN_LEFT</code>	<code>Motion_Ctrl(0, 0, speed * 3.0)</code>	Spin left in place (max 3 rad/s)
<code>MOTION_SPIN_RIGHT</code>	<code>Motion_Ctrl(0, 0, -speed * 3.0)</code>	Spin right in place (max 3 rad/s)
<code>MOTION_BRAKE</code>	<code>Motor_Stop(STOP_BRAKE)</code>	Brake (emergency stop)
<code>MOTION_STOP</code>	<code>Motor_Stop(STOP_COAST)</code>	Stop (coast)

Core functions:

- `Motion_Ctrl(float v_x, float v_y, float v_z)` — Controls the car's motion. The `v_y` parameter is invalid in the current differential chassis model and is kept only for interface extension
- `Motion_Get_Speed(car_motion_t *car)` — Gets the current linear velocity and angular velocity of the car
- `Motion_Ctrl_State(uint8_t state, float speed)` — Controls motion through predefined states for

quick invocation

- `Motion_Stop(uint8_t brake)` — Stops the car's motion

GPIO pins:

- S1: GPIO8
- S2: GPIO16

PWM parameters:

- Resolution: 1 MHz (1us/tick)
- Period: 20000 ticks (20ms, i.e., 50Hz)
- Pulse width range: 500 ~ 2500us (corresponding to angle -90° ~ +90°)
- S1 default angle: 0°, range: -90° ~ 90°
- S2 default angle: -45° (camera facing straight ahead), range: -90° ~ 20°

VI. Common Problems

Problem	Cause	Solution
The car does not drive straight	The encoder direction signs of the left and right wheels are inconsistent, or the wheel diameter has deviation	Adjust <code>MOTOR_ENCODER_SIGN_M1 / M2</code> or check whether the wheels are slipping
The car turns in the opposite direction	The sign definition of the angular velocity does not match expectations	Adjust the sign in the Wz calculation formula in <code>Motion_Get_Speed()</code>
Vy is always 0	The differential chassis model has no Y-direction degree of freedom; this is normal	No action needed
Inaccurate speed reading	The encoder parameters (pulse count/wheel diameter) do not match the actual hardware	Check the <code>MOTOR_ENCODER_CIRCLE</code> and <code>WHEEL_DIAMETER</code> configurations